MixBytes()

[DEPLOYMENT VERIFICATION REPORT]

Lido CSM 0x02

PREPARED FOR LIDO

REPORT DATE

October 08, 2026

[CONTENTS]

Table of Contents

1. Introduction 3

    1.1 Disclaimer 3

    1.2 Executive Summary 3

    1.3 Project Overview 5

2. About MixBytes 10

1.1 Disclaimer

The deployment verification makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

This report covers the deployment verification of the Community Staking Module 0x02 (CSM 0x02) contract set: eleven contracts (the `CSModule`, `Accounting`, `ParametersRegistry`, `FeeDistributor`, `FeeOracle`, `ValidatorStrikes`, and `ExitPenalties` implementations behind seven `OssifiableProxy` instances, plus the standalone `HashConsensus`, `Verifier`, `Ejector`, and `PermissionlessGate`) and nine shared libraries - twenty-seven deployed addresses in total, deployed from the `lidofinance/staking-modules` repository at release tag `v3.0` (commit `4d3de6658499e1c1774951780a97d7ae25ca18b8`, the final audited commit of the Certora and Statemind "Lido CSM v3 and CM v2" engagements). The CSM module code was not audited by MixBytes; this engagement verifies that the deployed bytecode is exactly the third-party-audited code and that the deployment configuration, initialization, and access-control handover are correct, consistent with the client's deployment artifacts (lidofinance/staking-modules#899).

The deployment verification was conducted over 1 working day via on-chain state inspection at block `26128512`, local recompilation of both the audit commit `4d3de6658499e1c1774951780a97d7ae25ca18b8` and the deployment git-ref `715c871d64a6771dacbc483528da58f1a3c734ad` with the repository's pinned Foundry configuration, byte-level bytecode comparison of all twenty-seven deployed addresses, cross-checks against the deployment manifest and access-control analysis of every role granted or revoked during deployment.

The verification vectors below were checked:

- **Bytecode identity vs the audited commit.** Each deployed implementation and library was compared byte-for-byte against a local Foundry build (solc `0.8.33`, EVM version `osaka`) at the deployment git-ref, and the deployment-ref build was in turn compared against a build of the audit commit `4d3de665`. All three are identical at every byte outside the constructor-immutable slots and the linked library addresses.
- **Proxy topology.** All seven proxy EIP-1967 implementation slots and admin slots were read on-chain and matched against the manifest and the Lido Aragon Agent.
- **Initialization safety.** Initializer state was read from the ERC-7201 namespaced storage slot on every proxy and every raw implementation: proxies are initialized, raw implementations carry the `_disableInitializers` mark (or an equivalent petrified version), so none can be initialization-front-run.
- **Constructor and initializer arguments.** Every creation transaction's constructor arguments and every `proxy__upgradeToAndCall` initializer calldata entry was decoded from the artifact's transaction log and matched against the deployment parameters and the on-chain getters.
- **Access control.** Every `grantRole` / `revokeRole` call in the deployment sequence was mapped and re-verified live on-chain, including confirmation that the deployment EOA holds no residual role on any contract.

- **Rollout state.** StakingRouter module registration, Easy Track factory registration, and CircuitBreaker pauser registration were read on-chain to establish the deployment's position in the governance sequence.

Out of scope of this verification is the security of the CSM 0x02 contract logic itself, which was reviewed by Certora and Statemind at commit `4d3de665`; MixBytes did not audit the module code. The Easy Track factory deployments bound to this module are covered by the separate report.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	CSM 0x02 - Deployment Verification
TYPE	Solidity
PLATFORM	EVM
TIMELINE	05.10.2026 - 07.10.2026

Scope

`src/CSModule.sol` `src/Accounting.sol` `src/ParametersRegistry.sol` `src/FeeDistributor.sol` `src/FeeOracle.sol` `src/Verifier.sol` `src/ValidatorStrikes.sol` `src/Ejector.sol` `src/ExitPenalties.sol` `src/PermissionlessGate.sol` `src/lib/base-oracle/HashConsensus.sol` `src/lib/proxy/OssifiableProxy.sol` `src/lib/AssetRecovererLib.sol` `src/lib/BondCurvesLib.sol` `src/lib/DepositQueueOps.sol` `src/lib/GeneralPenaltyLib.sol` `src/lib/NOAddresses.sol` `src/lib/NodeOperatorOps.sol` `src/lib/StakeTracker.sol` `src/lib/TopUpQueueOps.sol` `src/lib/WithdrawnValidatorLib.sol`

Versions Log

DATE	COMMIT HASH	NOTE
05.10.2026	`715c871d64a6771dacbc483528da58f1a3c734ad`	Deployment Verification Commit

The scope of this engagement is the verification of the twenty-seven mainnet deployments listed below against the codebase audited by Certora and Statemind at commit `4d3de6658499e1c1774951780a97d7ae25ca18b8`. No new code review was performed.

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
CSModule.sol (proxy)	0x792Cd25e4aE3578375031FB55e048E163A804F7B	Ethereum
CSModule.sol (implementation)	0xF66826CeFDD4F163f1e8bfB6c0b6808744264d46	Ethereum
Accounting.sol (proxy)	0x3696dDd942A9e156F5D4728505D1b9a32dCef900	Ethereum
Accounting.sol (implementation)	0x1e262530C7d142b2d1C113C11778294cc662874C	Ethereum
ParametersRegistry.sol (proxy)	0x5352b3d8274933c72dC774DccCD12B7301EdBfD7	Ethereum
ParametersRegistry.sol (implementation)	0x122E385434Eb5dD0a2a1EdF7b3b13e369C730E6e	Ethereum
FeeDistributor.sol (proxy)	0x122CdeDD0da630B555050976052C262EA25f8335	Ethereum
FeeDistributor.sol (implementation)	0x4e939B8c69e53900b2330A8C68e29953118Ca1C7	Ethereum
FeeOracle.sol (proxy)	0x0fB5EC09Cc975d8E1aF43063e51882798814f311	Ethereum
FeeOracle.sol (implementation)	0x607cBd562f83736BB258FC50b6d0ceB323471Ff9	Ethereum
ValidatorStrikes.sol (proxy)	0x89bd84481F7679732EADDF7d99d02ABD8309df65	Ethereum
ValidatorStrikes.sol (implementation)	0x855ead6778b1162bdB728e4613Af852cb2eb73c4	Ethereum
ExitPenalties.sol (proxy)	0xa96021Eff64E8FE927424C77902c56C4F17d10A5	Ethereum
ExitPenalties.sol (implementation)	0x5131f7f71C69b25AE320457d5543Ae0A26e4A7c2	Ethereum
HashConsensus.sol	0xd5a965FAab2d02D3cC2286A9da42d09F0F2aE210	Ethereum
Verifier.sol	0x69b4C32a43565e768794D41b4A265F86dE61b861	Ethereum
Ejector.sol	0x2EE500885870b020e84E86a09A5d26D1EEec3E5E	Ethereum
PermissionlessGate.sol	0x78e36353FE904c3685585D9F3204D4aAcb3Ad506	Ethereum
AssetRecovererLib.sol	0x37aDa408AE3c3992953688e2CCb9eE7a3dfdA902	Ethereum
BondCurvesLib.sol	0xC4511d09639e5E174506083443da230D39196323	Ethereum
DepositQueueOps.sol	0xb430AA6C70A352c2aaC9813AE049A210dB11aB41	Ethereum
GeneralPenaltyLib.sol	0xF05545ED71c60bBba6E73B6B70B15D4f5F22C0f4	Ethereum
NOAddresses.sol	0x9D9c8799189c797f6e2dA74F71aDF84492adA7D3	Ethereum
NodeOperatorOps.sol	0xDD42EE5D54A1822021782F3F455bb99fBC19499A	Ethereum
StakeTracker.sol	0xbb6E4Db18182d45038F91B9F1195291c206fd8d2	Ethereum
TopUpQueueOps.sol	0xdA104f5f2a18405fC7cCD6E0A7FEB5B824843606	Ethereum

FILE	ADDRESS	BLOCKCHAIN
WithdrawnValidatorLib.sol	0x3bf9674f062aF9BA94FdAe9Fcdf2D0001FFf0a3A	Ethereum

Deployment Verification Details

Deployment sequence. The deployment was executed from the EOA `0x1132510B0fBB75389359dFC5630C252CC950D408` in 87 transactions, all with successful status, all recorded in artifacts/mainnet/csm0x02/transactions.json with git-ref `715c871d64a6771dacbc483528da58f1a3c734ad`. Every transaction was decoded and classified: 20 contract creations (11 contracts, 9 libraries), 7 proxy creations, 5 proxy setup calls (`proxy__upgradeToAndCall` / `proxy__upgradeTo` / `proxy__changeAdmin`), 9 HashConsensus member additions, and 46 `grantRole` / `revokeRole` calls. No unexplained or out-of-pattern transaction exists in the sequence. Two inert 50-byte placeholder contracts (`0x3c57F1B5381F02F182006BC520Ab9183039c0185` , `0x27d9ba23cb75Ddf684FcAA9786282E3ab42f7445`) were used as interim proxy implementations during setup and remain on chain with no functionality.

Bytecode identity. The repository was built locally with Foundry at the deployment git-ref using the repository's own configuration at the audit commit `4d3de665`. For every deployment, the on-chain runtime code equals the local build at every byte outside the constructor-immutable slots, the linked library addresses, and the library self-address slots; and the audit-commit build equals the deployment-ref build at every byte outside those same slots (the only source difference between the two commits is `src/interfaces/IStakingRouter.sol` , an interface with no runtime code). The library addresses embedded in the deployed contracts equal the manifest's `ExternalLibraries` values, and each of the nine libraries embeds its own deployed address in its self-reference slot.

Proxy topology and initialization. All seven proxies are `OssifiableProxy` instances whose runtime code is byte-identical to the audited `OssifiableProxy.sol` . On-chain EIP-1967 slots: every implementation slot holds the manifest implementation address, and every admin slot holds the Lido Aragon Agent `0x3e40D73EB977Dc6a537aF587D48316feE66E9C8c` . Initializer state (ERC-7201 slot `0xf0c57e16840df040f15088dc2f81fe391c3923bec73e23a9662efc9c229c6a00`): CSModule, Accounting, ParametersRegistry, FeeDistributor initialized at version 3; ValidatorStrikes at version 1; ExitPenalties has no initializer by design; every raw implementation carries the `_disableInitializers` mark or the petrified version constant, so no raw implementation can be initialization-front-run. The FeeOracle was initialized at deployment like the others but tracks initialization through the base-oracle versioned pattern instead of the OpenZeppelin initializer: its proxy reports `getContractVersion() == 3` while its raw implementation is petrified at `type(uint256).max` by the `Versioned` constructor, so the oracle is initialized exactly once and cannot be initialization-front-run on the implementation. The module is deployed in the paused state (`isPaused() == true`), consistent with the launch flow in which the module is resumed as part of the governance activation.

Access control. `DEFAULT_ADMIN_ROLE` is held by the Aragon Agent on CSModule, Accounting, ParametersRegistry, FeeDistributor, FeeOracle, ValidatorStrikes, Verifier, Ejector, HashConsensus, and PermissionlessGate. The deployment EOA holds no role on any contract. The functional grants are:

CONTRACT	ROLE	HOLDER
CSModule, Accounting, Fee Oracle, Verifier, Ejector	PAUSE_ROLE	CircuitBreaker 0x6019CB557978296BA3C08a7B73225C0975DFB2F7 and ResealManager 0x7914b5a1539b97Bd0bbd155757F25FD79A522d24
CSModule, Accounting, Fee Oracle, Verifier, Ejector	RESUME_ROLE	ResealManager 0x7914b5a1539b97Bd0bbd155757F25FD79A522d24
CSModule	CREATE_NODE_OPERATOR_ROLE	PermissionlessGate 0x78e36353FE904c3685585D9F3204D4aAcb3Ad506
CSModule	VERIFIER_ROLE, REPORT_REGULAR_WITHDRAWN_VALIDATORS_ROLE	Verifier 0x69b4C32a43565e768794D41b4A265F86dE61b861
CSModule	REPORT_SLASHED_WITHDRAWN_VALIDATORS_ROLE, SETTLE_GENERAL_DELAYED_PENALTY_ROLE	Easy Track EVMScriptExecutor 0xFE5986E06210aC1eCC1aDCafc0cc7f8D63B3F977
CSModule	REPORT_GENERAL_DELAYED_PENALTY_ROLE, REWIND_TOP_UP_QUEUE_ROLE	CSM committee Safe 0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f
HashConsensus	MANAGE_MEMBERS_AND_QUORUM_ROLE	Aragon Agent 0x3e40D73EB977Dc6a537aF587D48316feE66E9C8c
ParametersRegistry	MANAGE_GENERAL_PENALTIES_AND_CHARGES_ROLE	CSM committee Safe 0xC52fC3081123073078698F1EAc2f1Dc7Bd71880f

The ResealManager at 0x7914b5a1539b97Bd0bbd155757F25FD79A522d24 was identified as the Emergency Protected Timelock's verified ResealManager contract, the standard Lido emergency pause/resume path; the CSM committee Safe remains the 4-of-6 Gnosis Safe v1.3.0 verified in the Easy Track factories engagement.

Configuration. Cross-bindings: CSModule.ACCOUNTING/PARAMETERS_REGISTRY/EXIT_PENALTIES/LIDO_LOCATOR, Accounting.MODULE/FEE_DISTRIBUTOR, FeeDistributor.ORACLE/ACCOUNTING, FeeOracle.getConsensusContract, ValidatorStrikes.ORACLE, Ejector.STRIKES, ExitPenalties.STRIKES (via constructor immutables), Verifier.MODULE/WITHDRAWAL_ADDRESS, and HashConsensus oracle binding all resolve to the manifest addresses, including the canonical LidoLocator 0xC1d0b3DE6792Bf6b4b37EccdcC24e45978Cfd2Eb and the Lido WithdrawalVault 0xB9D7934878B5FB9610B3fE8A5e441e8fad7E293f. Decoded constructor and initializer arguments match the manifest exactly, including: module type community-onchain-v1; the Verifier chain configuration (slotsPerEpoch 32, secondsPerSlot 12, genesis 1606824023, Capella slot 6209536, first supported and pivot slot 11649024, minWithdrawalRatio 9900, and the four generalized-index pairs); the Accounting default bond curve [(1, 32 ETH), (2, 30 ETH)] with bond lock period 4838400 seconds and charge penalty recipient set to the Aragon Agent; the CSModule top-up queue limit 16 (the value corrected in staking-modules#896); the FeeOracle consensus version 4 bound to HashConsensus 0xd5a965FAab2d02D3cC2286A9da42d09F0F2aE210; and ParametersRegistry defaults (key removal charge 0.02 ETH, bad performance penalty 0.258 ETH, general delayed penalty additional fine 0.1 ETH, reward share 10000 bp, keys limit unlimited). HashConsensus carries the nine manifest oracle members with quorum 5, epochs-per-frame 6300, and fast lane length 300 slots.

Rollout state. The StakingRouter 0xFdDf38947aFB03C621C71b06C9C70bce73f12999 holds staking modules [1, 2, 3, 4]; getStakingModule(5) reverts with StakingModuleUnregistered(). The CircuitBreaker's

pausable registry returns the zero address for all five CSM 0x02 pausable contracts, and none of the three Easy Track factories is registered. All three registrations are items of the pending DAO vote; the deployed contracts already hold the breaker's `PAUSE_ROLE`, so the breaker can pause them as soon as its own registry is updated. No misconfiguration window exists in the current state: the paused module cannot accept deposits, and the Easy Track motions cannot be created or enacted.

No inconsistency between the third-party audit reports and the deployed contracts was found: every fixed finding is present in the deployed bytecode (in the recommended or an equivalent verified form), every acknowledged finding is present exactly as documented.

One deliberate divergence from the proposal was identified and verified on-chain. The proposal listed `SET_BOND_CURVE_ROLE` among the roles to be assigned to the CSM Committee; the deployment intentionally does not grant it - staking-modules#898 ("do not grant set reset bond curve on CSM 0x02") removed the grant before the mainnet deployment. Verified on-chain: no address (including the Aragon Agent and the committee) holds either of the two Accounting bond-curve roles, `SET_BOND_CURVE_ROLE` or `MANAGE_BOND_CURVES_ROLE`, so the bond curve is frozen at the deployed default `[(1, 32 ETH), (2, 30 ETH)]`; the Aragon Agent, as `DEFAULT_ADMIN_ROLE` holder, can grant these roles later through a governance vote if per-operator curves are ever needed. This reduces the launch privilege surface (the VettedGate per-operator curve path is likewise absent at launch - the manifest records zero addresses for `VettedGate`, `VettedGateImpl`, and `VettedGateFactory`) and is a documented, client-authored deviation rather than a deployment defect. All other committee roles listed in the proposal (`REPORT_GENERAL_DELAYED_PENALTY_ROLE`, `MANAGE_GENERAL_PENALTIES_AND_CHARGES_ROLE`, `SETTLE_GENERAL_DELAYED_PENALTY_ROLE` via Easy Track, `REWIND_TOP_UP_QUEUE_ROLE`, and CircuitBreaker-based emergency pause) are granted exactly as proposed.

[2 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io